

SESSION 03

# JAVASCRIPT

Math Object, Spread Operator & Conditional Control Flow

---

## MATH OBJECT

random, floor, sqrt, pow, min, max

## SPREAD OPERATOR

unpack arrays into function arguments

## CONDITIONALS

if / else and ternary decisions

## PRACTICE

validation, grading, access and simulation

# Table of Contents

A page-by-page learning path. Every major concept begins on a fresh page.

| PAGE  | SECTION                                   | PAGE | SECTION                               |
|-------|-------------------------------------------|------|---------------------------------------|
| 03    | Session Overview                          | 04   | Math Object Overview                  |
| 05    | Math.random()                             | 06   | Math.floor() + Random Integer         |
| 07    | Math.sqrt()                               | 08   | Math.pow()                            |
| 09    | Math.min()                                | 10   | Math.max()                            |
| 11    | Spread Operator Overview                  | 12   | Spread with Math.max() / Math.min()   |
| 13    | Conditional Statements Overview           | 14   | if / else Example                     |
| 15    | Ternary Operator                          | 16   | if / else vs Ternary                  |
| 17    | Practice Q3 - Voting Eligibility          | 18   | Practice Q4 - Largest of Two          |
| 19    | Practice Q5 - Largest of Three            | 20   | Practice Q6 - Divisibility by 3 and 5 |
| 21    | Practice Q7 - Divisibility by 5 and 10    | 22   | Practice Q8 - Student Grading         |
| 23    | Practice Q9 - Age Category                | 24   | Practice Q10 - Range Check            |
| 25    | Practice Q11 - Username & Password        | 26   | Practice Q12 - Restricted Access      |
| 27    | Practice Q13 - Positive and Even          | 28   | Practice Q14 - Negative or Odd        |
| 29    | Practice Q15 - Temperature Classification | 30   | Practice Q16 - Ticket Type            |
| 31    | Practice Q17 - Admin Panel                | 32   | Practice Q18 - All Subjects Passed    |
| 33    | Practice Q19 - Positive / Negative / Zero | 34   | Practice Q20 - ATM Simulation         |
| 35    | Additional Practice 21-25                 | 36   | Additional Practice 26-30             |
| 37    | Challenge Questions                       | 38   | Answer Key - Source Practice          |
| 39    | Answer Key - Additional & Challenge       | 40   | Interview Revision                    |
| 41    | Quick Revision Sheet                      | 42   | Session Summary                       |
| 43-60 | Clean Session Code Appendix               |      |                                       |

# Session Overview

What this session adds after operators and basic conditions.

## Session 3 builds practical control-flow skills around math utilities, array expansion and decisions.

This session introduces three connected ideas: the built-in **Math** object, the **spread operator** and **conditional control flow**.

### Learning path

Math helpers -> unpack values -> make decisions -> solve practice problems.

| Area            | What you learn                                                                        |
|-----------------|---------------------------------------------------------------------------------------|
| Math Object     | Generate random values, round down, find square roots, powers, minimums and maximums. |
| Spread Operator | Expand an array into individual arguments for a function call.                        |
| Conditionals    | Run different code depending on whether a condition is true or false.                 |
| Practice        | Apply the ideas to validation, grading, access checks, ranges and simple simulations. |

### Session focus

The notes keep the source terminology and examples, while presenting each major concept on its own page for focused study.

# Math Object

A built-in toolbox for common mathematical operations.

---

## The Math object exposes pre-made mathematical constants and static methods.

### Simple words

A built-in toolbox in JavaScript that gives you pre-made math formulas and utility functions such as rounding, generating random numbers and finding powers.

### Interview prep

A static standard built-in namespace object (not a constructor function) that exposes properties and static methods for mathematical constants and functions.

### Code

```
Math.random();  
Math.floor(value);  
Math.sqrt(value);  
Math.pow(base, exponent);  
Math.min(a, b, c);  
Math.max(a, b, c);
```

### Remember

Math methods are called on the Math object. They are ready-made tools; you do not create a Math instance with new.

# Math.random()

Generate a pseudo-random floating-point value.

---

**Math.random() returns a value from 0 (inclusive) up to, but not including, 1.**

## Code

```
let randomVal = Math.random();  
console.log("Random Value (0 to 1):", randomVal);
```

## How it works

The result is a floating-point number in the range [0, 1). The lower bound can be exactly 0, while 1 is not returned.

## Example output

Random Value (0 to 1): 0.538214...

## Important

The exact value changes on each execution. Use Math.random() when the problem needs a pseudo-random value, not a fixed constant.

# Math.floor() + Random Integer

Round down and build a random integer from 0 to 9.

**Math.floor()** rounds a decimal down to the nearest whole number. Combined with **Math.random() \* 10**, it produces an integer from 0 through 9.

## Code

```
let randomInt = Math.floor(Math.random() * 10);  
console.log("Random Integer (0 to 9):", randomInt);
```

## Step by step

**Math.random()** produces a value from 0 up to 1. Multiplying by 10 produces a value from 0 up to 10. **Math.floor()** removes the decimal part by rounding downward.

## Example output

Random Integer (0 to 9): 7

## Remember

**Math.floor()** always moves toward the lower whole number. This is different from normal rounding.

# Math.sqrt()

Calculate the square root of a number.

---

**Math.sqrt(x) returns the square root of x.**

## Code

```
let squareRoot = Math.sqrt(16);  
console.log("Square Root of 16:", squareRoot);
```

## Expected output

Square Root of 16: 4

## Concept

The square root of 16 is 4 because  $4 * 4 = 16$ .

## Use case

Useful when a problem asks you to calculate the root of a numeric value without manually implementing the operation.

# Math.pow()

Raise a base to a power.

**Math.pow(base, exponent)** calculates base raised to the given exponent.

## Code

```
let powerVal = Math.pow(2, 5);  
console.log("2 raised to power 5:", powerVal);
```

## Expected output

2 raised to power 5: 32

| Expression     | Meaning                              | Result |
|----------------|--------------------------------------|--------|
| Math.pow(2, 5) | 2 raised to the 5th power            | 32     |
| 2 ** 5         | Equivalent exponentiation expression | 32     |

## Source note

The session explicitly states that Math.pow(base, exponent) is equivalent to the exponentiation operator \*\* for the example shown.

# Math.min()

Find the smallest value among multiple arguments.

---

**Math.min(...values) returns the smallest numeric value passed as an argument.**

## Code

```
let minVal = Math.min(10, 20, 30, 32, 44, 2, 1, 5, 7, 4);  
console.log("Minimum Value:", minVal);
```

## Expected output

Minimum Value: 1

## Important

Math.min() expects separate numeric arguments. An array needs to be expanded with the spread operator before being passed to Math.min().

# Math.max()

Find the largest value among multiple arguments.

---

**Math.max(...values)** returns the largest numeric value passed as an argument.

## Code

```
let maxVal = Math.max(10, 20, 30, 32, 44, 2, 1, 5, 7, 4);  
console.log("Maximum Value:", maxVal);
```

## Expected output

Maximum Value: 44

## Connection

Math.max() becomes especially useful with the spread operator when the values are already stored in an array.

# Spread Operator (...)

Expand iterable values into individual items.

---

**The spread operator unpacks an array so its elements can be used as individual arguments in a function call.**

## Simple words

Unpacks an array of values into individual standalone items.

## Interview prep

An ES6 syntax feature that expands an iterable, such as an Array, into individual arguments in a function call or elements in an array literal.

## Code

```
const numberList = [10, 20, 30];  
  
Math.max(...numberList);
```

## Mental model

Read ...numberList as: "take the items inside numberList and pass them one by one here."

# Spread with Math.max() / Math.min()

Use an array with functions that expect separate arguments.

**Spread solves the mismatch between an array value and a function call that expects individual arguments.**

## Code

```
let numberList = [10, 20, 30, 32, 44, 2, 1, 5, 7, 4];

let arrayMax = Math.max(...numberList);
console.log("Max using Spread Operator:", arrayMax);

let arrayMin = Math.min(...numberList);
console.log("Min using Spread Operator:", arrayMin);
```

## Expected output

```
Max using Spread Operator: 44
Min using Spread Operator: 1
```

## Key idea

Without spread, numberList is one array argument. With spread, the call receives 10, 20, 30, ... as individual arguments.

# Conditional Statements

Decision-making control flow with if / else if / else.

**Conditional statements choose which block of code to run based on whether a condition is true or false.**

## Simple words

Decision-making statements used to run different blocks of code depending on whether a condition is true or false.

## Interview prep

Control flow statements that evaluate boolean expressions to determine which branch of code to execute during runtime.

## Code

```
if (condition) {  
    // runs when condition is true  
} else if (anotherCondition) {  
    // runs when the next condition is true  
} else {  
    // runs when earlier conditions are false  
}
```

## Flow

Condition -> choose a branch -> run that block. Once a matching branch is selected, later else-if / else blocks are skipped.

# if / else Example

Check voting eligibility using a condition.

---

**The if block runs for an age of 18 or above; otherwise the else block runs.**

## Code

```
let checkAge = 18;

if (checkAge >= 18) {
  console.log("You are eligible to vote");
} else {
  console.log("You are not eligible to vote");
}
```

## Expected output

You are eligible to vote

## How to read it

checkAge >= 18 is the condition. If it evaluates to true, the first block executes. Otherwise, the else block executes.

## Important

A condition used in if / else evaluates to a Boolean decision: true or false.

# Ternary Operator

A compact conditional expression that returns a value.

**The ternary operator is an inline form of a two-way condition.**

## Simple words

A one-line shortcut for writing an if-else condition: `condition ? trueResult : falseResult`.

## Interview prep

An inline conditional expression accepting three operands, returning a value based on boolean evaluation.

## Code

```
let ternaryAge = 18;
let voteEligibility = (ternaryAge >= 18)
  ? "You are eligible to vote"
  : "You are not eligible to vote";

console.log("Ternary Result:", voteEligibility);
```

## Expected output

Ternary Result: You are eligible to vote

## Pattern

`condition ? valueWhenTrue : valueWhenFalse`

# if / else vs Ternary

Same decision idea, two different styles.

**Use if / else for multi-step or multi-branch logic; use ternary when a simple condition selects between two values.**

| Approach  | Pattern                             | Best fit in this session                  |
|-----------|-------------------------------------|-------------------------------------------|
| if / else | if (condition) { ... } else { ... } | Running different blocks of code.         |
| ternary   | condition ? trueValue : falseValue  | Choosing one value from two alternatives. |

## Source example

The session uses `ternaryAge >= 18` to assign `voteEligibility` to one of two strings.

## Remember

The ternary operator is an expression that produces a value. The source describes it as a one-line shortcut for if-else.

# Practice Q3 - Voting Eligibility with Validation

Source practice exercise

**Check that the age is valid before deciding voting eligibility.**

## Code

```
let userAge = 28;

if (userAge < 0) {
  console.log("Q3: Age must be a positive number.");
} else if (userAge >= 18) {
  console.log("Q3: Eligible to Vote");
} else {
  console.log("Q3: Not eligible to vote");
}
```

## How it works

The first condition rejects negative ages. If the age is valid and at least 18, the second branch runs. Otherwise the final else branch runs.

## Expected output

Q3: Eligible to Vote

## Remember

This example combines validation with if / else if / else.

# Practice Q4 - Largest of Two Numbers

Source practice exercise

---

**Use a ternary expression to select the larger of two numbers.**

## Code

```
let n1 = 23;  
let n2 = 34;  
  
let largerNum = (n1 > n2) ? n1 : n2;  
  
console.log("Q4: Larger number is ->", largerNum);
```

## How it works

If n1 is greater than n2, largerNum receives n1. Otherwise it receives n2.

## Expected output

Q4: Larger number is -> 34

# Practice Q5 - Largest of Three Numbers

Source practice exercise

**Use Math.max() to find the largest value among three numbers.**

## Code

```
let num1 = 23;  
let num2 = 34;  
let num3 = 56;  
  
let largestOfThree = Math.max(num1, num2, num3);  
  
console.log("Q5: Largest of three numbers ->", largestOfThree);
```

## How it works

Math.max() compares all supplied numeric arguments and returns the highest value.

## Expected output

Q5: Largest of three numbers -> 56

# Practice Q6 - Divisibility by 3 and 5

Source practice exercise

**Use modulus plus logical operators to decide whether a number is divisible by both, only one, or neither.**

## Code

```
let testNumber1 = 15;
const divA1 = 3;
const divB1 = 5;

if (testNumber1 % divA1 === 0 && testNumber1 % divB1 === 0) {
  console.log("Q6: Number is divisible by BOTH 3 and 5.");
} else if (testNumber1 % divA1 === 0 || testNumber1 % divB1 === 0) {
  console.log("Q6: Number is divisible by ONLY ONE of the numbers.");
} else {
  console.log("Q6: Number is not divisible by any of the numbers.");
}
```

## How it works

A remainder of 0 means the number is divisible by the divisor. && checks whether both divisibility tests are true; || checks whether at least one is true.

## Expected output

Q6: Number is divisible by BOTH 3 and 5.

## Practice Q7 - Divisibility by 5 and 10

Source practice exercise

---

**Apply the same logical pattern to two different divisors.**

### Code

```
let testNumber2 = 15;
const fNum1 = 5;
const sNum1 = 10;

if (testNumber2 % fNum1 === 0 && testNumber2 % sNum1 === 0) {
  console.log("Q7: Number is divisible by BOTH 5 and 10.");
} else if (testNumber2 % fNum1 === 0 || testNumber2 % sNum1 === 0) {
  console.log("Q7: Number is divisible by ONLY ONE of the numbers (5 or 10).");
} else {
  console.log("Q7: Number is not divisible by any of the numbers.");
}
```

### How it works

The same modulus-plus-logical pattern is reused with 5 and 10. For 15, only division by 5 leaves remainder 0.

### Expected output

Q7: Number is divisible by ONLY ONE of the numbers (5 or 10).

# Practice Q8 - Student Grading System

Source practice exercise

**Validate marks first, then assign the grade using nested conditions.**

## Code

```
let marks = 99;
let total = 100;
let studentGrade = "";

if (marks >= 0 && marks <= total) {
  if (marks >= 90) {
    studentGrade = 'A+';
  } else if (marks >= 70) {
    studentGrade = 'A';
  } else if (marks >= 40) {
    studentGrade = 'B';
  } else if (marks >= 30) {
    studentGrade = 'C';
  } else {
    console.log("Q8: Student has Failed.");
  }

  if (studentGrade) {
    console.log("Q8: Student Grade is =>", studentGrade);
  }
} else if (marks < 0) {
  console.log("Q8: Marks must be positive.");
} else {
  console.log("Q8: Marks cannot be greater than total marks!");
}
```

## How it works

The outer condition validates the marks range. Inside it, the nested if / else-if chain chooses the grade. For 99 marks, the first grade rule matches.

## Expected output

```
Q8: Student Grade is => A+
```

# Practice Q9 - Categorize Person by Age

Source practice exercise

**Classify a person as child, teenager or adult using else-if branches.**

## Code

```
let personAge = 0;

if (personAge <= 0) {
  console.log("Q9: Age must be a positive integer.");
} else if (personAge < 12) {
  console.log("Q9: Child");
} else if (personAge <= 18) {
  console.log("Q9: Teenager");
} else {
  console.log("Q9: Adult");
}
```

## How it works

The first condition validates the age. Because personAge is 0, the validation branch is selected before the category checks.

## Expected output

Q9: Age must be a positive integer.

# Practice Q10 - Range Check

Source practice exercise

---

**Check whether a number is between 10 and 50, inclusive.**

## Code

```
let n3 = 34;

if (n3 >= 10 && n3 <= 50) {
  console.log("Q10:", n3, "is within the range of 10 to 50");
} else {
  console.log("Q10:", n3, "is outside the range of 10 to 50");
}
```

## How it works

The number must satisfy both bounds. && makes the two comparisons one combined condition.

## Expected output

Q10: 34 is within the range of 10 to 50

# Practice Q11 - Username and Password

Source practice exercise

---

**Use strict equality and logical AND to validate two credential fields.**

## Code

```
let inputUsername = "admin_user";
let inputPassword = "SecurePassword123";

const validUsername = "admin_user";
const validPassword = "SecurePassword123";

if (inputUsername === validUsername && inputPassword === validPassword) {
  console.log("Q11: Username and password are correct.");
} else {
  console.log("Q11: Invalid username or password.");
}
```

## How it works

Both the username and password comparisons must be true. Strict equality is used so the compared values and types must match.

## Expected output

Q11: Username and password are correct.

# Practice Q12 - Restricted Area Access

Source practice exercise

---

**Grant access only when the visitor is 18+ and has a valid ID.**

## Code

```
let visitorAge = 20;
let hasValidID = true;

if (visitorAge >= 18 && hasValidID) {
  console.log("Q12: Access granted to restricted area.");
} else {
  console.log("Q12: Access denied. Must be 18+ and possess an ID.");
}
```

## How it works

The access rule combines an age comparison and a Boolean value. && requires both conditions to be true.

## Expected output

Q12: Access granted to restricted area.

# Practice Q13 - Positive and Even

Source practice exercise

---

## Check whether a number is both positive and even.

### Code

```
let checkNumPositiveEven = 14;

if (checkNumPositiveEven > 0 && checkNumPositiveEven % 2 === 0) {
  console.log("Q13:", checkNumPositiveEven, "is Positive and Even.");
} else {
  console.log("Q13:", checkNumPositiveEven, "is NOT both Positive and Even.");
}
```

### How it works

The first condition checks positivity. The second uses modulus to check whether the remainder after division by 2 is zero.

### Expected output

Q13: 14 is Positive and Even.

# Practice Q14 - Negative or Odd

Source practice exercise

---

## Check whether a number is negative or odd.

### Code

```
let checkNumNegOdd = -7;

if (checkNumNegOdd < 0 || checkNumNegOdd % 2 !== 0) {
  console.log("Q14:", checkNumNegOdd, "is Negative or Odd.");
} else {
  console.log("Q14:", checkNumNegOdd, "is neither Negative nor Odd (It is Positive and Even).");
}
```

### How it works

|| needs only one side to be true. -7 is negative, and it is also odd, so the first branch runs.

### Expected output

Q14: -7 is Negative or Odd.

# Practice Q15 - Temperature Classification

Source practice exercise

---

**Classify temperature into hot, normal or cold using ordered conditions.**

## Code

```
let tempCelsius = 28;

if (tempCelsius > 35) {
  console.log("Q15: Temperature Classification -> Hot");
} else if (tempCelsius >= 15) {
  console.log("Q15: Temperature Classification -> Normal");
} else {
  console.log("Q15: Temperature Classification -> Cold");
}
```

## How it works

The first matching branch wins. 28 is not greater than 35, but it is at least 15, so the normal branch runs.

## Expected output

```
Q15: Temperature Classification -> Normal
```

# Practice Q16 - Ticket Type via Ternary

Source practice exercise

---

**Use a ternary expression to select a child or adult ticket.**

## Code

```
let ticketAge = 10;  
let ticketType = (ticketAge < 12) ? "Child Ticket" : "Adult Ticket";  
  
console.log("Q16: Ticket Assigned ->", ticketType);
```

## How it works

If ticketAge is below 12, the expression returns Child Ticket; otherwise it returns Adult Ticket.

## Expected output

```
Q16: Ticket Assigned -> Child Ticket
```

# Practice Q17 - Admin Panel Access

Source practice exercise

---

**Allow access only when the user is both logged in and an admin.**

## Code

```
let isLoggedIn = true;
let isAdmin = true;

if (isLoggedIn && isAdmin) {
  console.log("Q17: Access granted to Admin Panel.");
} else {
  console.log("Q17: Access denied. User must be logged in AND have Admin status.");
}
```

## How it works

Two Boolean conditions are combined with &&. Both are true in the supplied example.

## Expected output

Q17: Access granted to Admin Panel.

# Practice Q18 - Passed All Subjects

Source practice exercise

---

**Check three subjects with the same passing requirement.**

## Code

```
let mathMarks = 45;
let scienceMarks = 38;
let englishMarks = 50;
const passingMarks = 35;

if (mathMarks >= passingMarks && scienceMarks >= passingMarks && englishMarks >= passingMarks) {
  console.log("Q18: Student has passed all subjects.");
} else {
  console.log("Q18: Student has failed one or more subjects.");
}
```

## How it works

All three comparisons must be true. Since each mark is at least 35, the pass branch runs.

## Expected output

Q18: Student has passed all subjects.

# Practice Q19 - Positive, Negative, or Zero

Source practice exercise

**Use a three-branch conditional to classify zero separately from positive and negative values.**

## Code

```
let valueToCheck = 0;

if (valueToCheck > 0) {
  console.log("Q19:", valueToCheck, "is Positive.");
} else if (valueToCheck < 0) {
  console.log("Q19:", valueToCheck, "is Negative.");
} else {
  console.log("Q19: Number is Zero.");
}
```

## How it works

The first two comparisons are false for zero, so the final else branch executes.

## Expected output

Q19: Number is Zero.

# Practice Q20 - ATM Withdrawal Simulation

Source practice exercise

**Validate the PIN first, then validate the withdrawal amount and update the balance.**

## Code

```
let correctPin = 4567;
let withdrawalAmount = 10000;
let accountBalance = 50000;

if (typeof prompt !== "undefined") {
  let enteredPin = Number(prompt("Enter PIN:"));

  if (enteredPin === correctPin) {
    console.log("Q20: PIN is correct.");

    if (withdrawalAmount > 0 && withdrawalAmount <= accountBalance) {
      accountBalance -= withdrawalAmount;
      console.log("Q20: Withdrawal successful! Withdrawn Amount:", withdrawalAmount);
      console.log("Q20: Remaining Balance:", accountBalance);
    } else if (withdrawalAmount > accountBalance) {
      console.log("Q20: Insufficient Balance!");
    } else {
      console.log("Q20: Invalid withdrawal amount!");
    }
  } else {
    console.log("Q20: Pin is Incorrect. Cannot Withdraw.");
  }
}
```

## How it works

The source checks whether `prompt` exists before requesting input. After the PIN is converted to a number, strict equality validates it. A nested condition then checks the withdrawal amount against the balance.

## Expected output

Browser example with correct PIN 4567: PIN is correct; withdrawal successful; remaining balance = 40000.

## Important

The source intentionally guards `prompt()` with `typeof prompt !== "undefined"` so the example does not fail in environments where `prompt` is unavailable.

# Additional Practice 21-25

Additional practice

## Apply Math methods, spread and basic conditionals.

Supplemental practice based on Session 3 concepts.

### Q21 - Random Integer

Use `Math.random()` and `Math.floor()` to generate a random integer from 1 to 6, similar to a dice roll.

#### Suggested solution

```
let dice = Math.floor(Math.random() * 6) + 1;
console.log("Q21: Dice value ->", dice);
```

#### Explanation

`Math.random() * 6` gives a value from 0 up to 6. Adding 1 after floor shifts the integer range to 1 through 6.

#### Example output

Example: Q21: Dice value -> 4

### Q22 - Square Root

Calculate the square root of 81 using the Math object.

#### Suggested solution

```
let value = 81;
let root = Math.sqrt(value);
console.log("Q22: Square root ->", root);
```

#### Explanation

`Math.sqrt(81)` returns the number whose square is 81.

#### Example output

Q22: Square root -> 9

### Q23 - Largest from an Array

Store five numbers in an array and use spread with `Math.max()` to find the largest.

#### Suggested solution

```
let values = [12, 48, 7, 19, 33];
let largest = Math.max(...values);
console.log("Q23: Largest ->", largest);
```

### Explanation

The spread operator turns the array into individual Math.max() arguments.

### Example output

Q23: Largest -> 48

## Q24 - Age Category

Use if / else if / else to print Child, Teenager or Adult for a given age.

### Suggested solution

```
let age = 16;

if (age < 12) {
  console.log("Q24: Child");
} else if (age <= 18) {
  console.log("Q24: Teenager");
} else {
  console.log("Q24: Adult");
}
```

### Explanation

The first matching condition determines the category.

### Example output

Q24: Teenager

## Q25 - Eligible or Not

Use the ternary operator to assign a message based on whether a person is at least 18.

### Suggested solution

```
let age = 21;
let result = age >= 18 ? "Eligible" : "Not eligible";
console.log("Q25:", result);
```

### Explanation

The condition age >= 18 is true, so the first value is selected.

### Example output

Q25: Eligible

# Additional Practice 26-30

Additional practice

## Continue with Math, spread, ranges and conditional decisions.

Supplemental practice based on Session 3 concepts.

### Q26 - Minimum Value from Array

Use spread with Math.min() to find the smallest value in an array.

#### Suggested solution

```
let values = [23, 4, 18, 11, 9];
let smallest = Math.min(...values);
console.log("Q26: Smallest ->", smallest);
```

#### Explanation

Spread supplies the five array elements as separate arguments to Math.min().

#### Example output

Q26: Smallest -> 4

### Q27 - Number Range

Check whether a number lies between 20 and 40 inclusive.

#### Suggested solution

```
let n = 37;

if (n >= 20 && n <= 40) {
  console.log("Q27: Inside range");
} else {
  console.log("Q27: Outside range");
}
```

#### Explanation

Both boundary comparisons must be true.

#### Example output

Q27: Inside range

### Q28 - Power Calculation

Use Math.pow() to calculate 3 raised to the power 4.

#### Suggested solution

```
let result = Math.pow(3, 4);  
console.log("Q28: Power ->", result);
```

#### Explanation

Math.pow(base, exponent) returns base raised to exponent.

#### Example output

Q28: Power -> 81

### Q29 - Access Check

Grant access only when the user is 18+ and has a valid ID.

#### Suggested solution

```
let age = 22;  
let hasID = true;  
  
if (age >= 18 && hasID) {  
  console.log("Q29: Access granted");  
} else {  
  console.log("Q29: Access denied");  
}
```

#### Explanation

Both the age condition and the Boolean ID condition must be true.

#### Example output

Q29: Access granted

### Q30 - Temperature Label

Classify a temperature above 35 as Hot, 15 to 35 as Normal, otherwise Cold.

#### Suggested solution

```
let temp = 12;  
  
if (temp > 35) {  
  console.log("Q30: Hot");  
} else if (temp >= 15) {  
  console.log("Q30: Normal");  
} else {  
  console.log("Q30: Cold");  
}
```

#### Explanation

The conditions are checked from top to bottom until one is true.

### Example output

Q30: Cold

# Challenge Questions

Practice prediction and reasoning without running the code.

---

**These questions focus on tracing the same concepts used in Session 3.**

## Challenge 1 - Predict the random range

What is the range of values produced by `Math.floor(Math.random() * 10)`?

### Expected reasoning

Expected idea: integers 0 through 9.

## Challenge 2 - Spread tracing

What values are received by `Math.max(...[8, 2, 17, 5])`?

### Expected reasoning

Expected answer: 8, 2, 17 and 5 are passed as individual arguments; result = 17.

## Challenge 3 - Nested grade logic

What grade does the source Q8 assign to marks = 72?

### Expected reasoning

Expected answer: A.

## Challenge 4 - Condition combination

For age = 20 and hasValidID = false, what does Q12 print and why?

### Expected reasoning

Expected answer: Access denied because `&&` requires both conditions to be true.

## Challenge 5 - Branch order

For tempCelsius = 40 in Q15, which branch runs?

### Expected reasoning

Expected answer: Hot, because `tempCelsius > 35` is true.

# Answer Key - Source Practice

Expected outcomes for Q3-Q20 from the supplied session.

**Use this page to check your own reasoning after attempting the source exercises.**

| Question | Expected result                                                                  |
|----------|----------------------------------------------------------------------------------|
| Q3       | Eligible to Vote                                                                 |
| Q4       | 34                                                                               |
| Q5       | 56                                                                               |
| Q6       | Divisible by BOTH 3 and 5                                                        |
| Q7       | Divisible by ONLY ONE of 5 or 10                                                 |
| Q8       | A+                                                                               |
| Q9       | Age must be a positive integer                                                   |
| Q10      | 34 is within the range of 10 to 50                                               |
| Q11      | Username and password are correct                                                |
| Q12      | Access granted                                                                   |
| Q13      | 14 is Positive and Even                                                          |
| Q14      | -7 is Negative or Odd                                                            |
| Q15      | Normal                                                                           |
| Q16      | Child Ticket                                                                     |
| Q17      | Access granted to Admin Panel                                                    |
| Q18      | Student has passed all subjects                                                  |
| Q19      | Number is Zero                                                                   |
| Q20      | Depends on entered PIN; with 4567, withdrawal succeeds and balance becomes 40000 |

## Note on Q20

The ATM example requires browser prompt input. The supplied values define the correct PIN as 4567, the withdrawal amount as 10000 and the starting balance as 50000.

# Answer Key - Additional & Challenge

Supplemental answers and reasoning.

**These answers support self-checking after the additional practice and challenge section.**

| Question | Expected output / result      |
|----------|-------------------------------|
| Q21      | Example: Q21: Dice value -> 4 |
| Q22      | Q22: Square root -> 9         |
| Q23      | Q23: Largest -> 48            |
| Q24      | Q24: Teenager                 |
| Q25      | Q25: Eligible                 |

| Question | Expected output / result |
|----------|--------------------------|
| Q26      | Q26: Smallest -> 4       |
| Q27      | Q27: Inside range        |
| Q28      | Q28: Power -> 81         |
| Q29      | Q29: Access granted      |
| Q30      | Q30: Cold                |

## Challenge quick answers

1) 0-9; 2) 17; 3) A; 4) Access denied; 5) Hot.

# Interview Revision

Short answers for common Session 3 questions.

---

**Keep these answers concise enough to revise before an interview or viva.**

**What is the Math object?**

A built-in namespace object that provides mathematical constants and static methods.

**What does Math.random() return?**

A pseudo-random floating-point number from 0 inclusive to 1 exclusive.

**What does Math.floor() do?**

It rounds a number down to the nearest whole number.

**What does the spread operator do?**

It expands an iterable such as an array into individual arguments in a function call.

**What is a conditional statement?**

A control-flow construct that chooses which branch of code to execute based on a condition.

**What is the ternary operator?**

An inline conditional expression with three operands that returns one of two values.

# Quick Revision Sheet

One-page cheat sheet for Session 3.

**Use this page for fast recall.**

| Concept           | Quick reminder                      |
|-------------------|-------------------------------------|
| Math.random()     | $0 \leq \text{value} < 1$           |
| Math.floor()      | Round down                          |
| Math.sqrt(x)      | Square root of x                    |
| Math.pow(a,b)     | a raised to b                       |
| Math.min(a,b,...) | Smallest argument                   |
| Math.max(a,b,...) | Largest argument                    |
| Spread (...)      | Unpack iterable values              |
| if / else         | Choose a code branch                |
| ternary           | condition ? trueValue : falseValue  |
| &&                | Both conditions must be true        |
|                   | At least one condition must be true |
| % === 0           | Common divisibility pattern         |

## Core flow

Math tools -> spread arrays into arguments -> use conditions -> solve practical problems.

# Session Summary

What you should be able to do after Session 3.

---

## Session 3 connects built-in Math tools with data expansion and conditional decision-making.

- Use `Math.random()` to generate pseudo-random floating-point values.
- Build random integers with `Math.floor()`.
- Calculate square roots and powers with `Math.sqrt()` and `Math.pow()`.
- Find minimum and maximum values with `Math.min()` and `Math.max()`.
- Use the spread operator to pass array elements as individual function arguments.
- Write `if / else if / else` decision logic.
- Use the ternary operator when a condition selects between two values.
- Combine comparisons, modulus and logical operators inside practical conditions.
- Trace validation, grading, access and simulation-style problems.

### Ready for the next session?

You should now be able to read a small problem, identify the needed Math method or condition, and translate the logic into JavaScript step by step.



# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
/* =====  
SESSION 3: MATH OBJECT, SPREAD OPERATOR & CONDITIONAL CONTROL FLOW  
===== */  
  
console.log("=== SESSION 3: MATH OBJECT, SPREAD OPERATOR & CONDITIONALS ===");  
  
// =====  
// 1. BUILT-IN MATH OBJECT METHODS  
// =====  
  
/*  
 * CONCEPT: The Math Object  
 *  
 * Simple Words Definition:  
   A built-in toolbox in JavaScript that gives you pre-made math formulas  
   and utility functions (like rounding, generating random numbers, and finding powers).  
 *  
 * Interview Prep Definition:  
   A static standard built-in namespace object (not a constructor function)  
   that exposes properties and static methods for mathematical constants and functions.  
 */
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
*/

// Math.random() -> Generates a pseudo-random floating-point number between 0 (inclusive) and 1 (exclusive)
let randomVal = Math.random();
console.log("Random Value (0 to 1):", randomVal);

// Combining Math.random() and Math.floor() to generate a random integer from 0 to 9
// Math.floor() rounds a decimal DOWN to the nearest whole integer
let randomInt = Math.floor(Math.random() * 10);
console.log("Random Integer (0 to 9):", randomInt);

// Math.sqrt(x) -> Calculates the square root of a given number x
let squareRoot = Math.sqrt(16);
console.log("Square Root of 16:", squareRoot); // 4

// Math.pow(base, exponent) -> Calculates base raised to the power of exponent (equivalent to base ** exponent)
let powerVal = Math.pow(2, 5);
console.log("2 raised to power 5:", powerVal); // 32

// Math.min(...values) -> Returns the smallest value among the provided numeric arguments
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
let minVal = Math.min(10, 20, 30, 32, 44, 2, 1, 5, 7, 4);
console.log("Minimum Value:", minVal); // 1

// Math.max(...values) -> Returns the largest value among the provided numeric arguments
let maxVal = Math.max(10, 20, 30, 32, 44, 2, 1, 5, 7, 4);
console.log("Maximum Value:", maxVal); // 44

// =====
// 2. SPREAD OPERATOR WITH MATH METHODS
// =====

/*
 * CONCEPT: Spread Operator (...) with Functions
 *
 * Simple Words Definition:
 * Unpacks an array of values into individual standalone items.
 *
 * Interview Prep Definition:
 * An ES6 syntax feature that expands an iterable (like an Array) into individual
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
    arguments in a function call or elements in an array literal.
    */

let numberList = [10, 20, 30, 32, 44, 2, 1, 5, 7, 4];

// Math.max/min expect individual comma-separated arguments, not a single array instance.
// Using '...numberList' unpacks the array elements into individual parameters.
let arrayMax = Math.max(...numberList);
console.log("Max using Spread Operator:", arrayMax); // 44

let arrayMin = Math.min(...numberList);
console.log("Min using Spread Operator:", arrayMin); // 1

// =====
// 3. CONDITIONAL STATEMENTS (if / else & Ternary)
// =====

/*
 * CONCEPT: Conditional Statements (if / else if / else)
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
*  
* Simple Words Definition:  
* Decision-making statements used to run different blocks of code depending on whether a condition is true or false.  
*  
* Interview Prep Definition:  
* Control flow statements that evaluate boolean expressions to determine which branch of code to execute during runtime.  
*/  
  
let checkAge = 18;  
  
// Standard if-else statement  
if (checkAge >= 18) {  
  console.log("You are eligible to vote");  
} else {  
  console.log("You are not eligible to vote");  
}  
  
/*  
* CONCEPT: Ternary Operator  
*/
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
* Simple Words Definition:
  A one-line shortcut for writing an if-else condition: (condition ? trueResult : falseResult).
*
* Interview Prep Definition:
  An inline conditional expression accepting three operands, returning a value based on boolean evaluation.
*/

let ternaryAge = 18;
let voteEligibility = (ternaryAge >= 18) ? "You are eligible to vote" : "You are not eligible to vote";
console.log("Ternary Result:", voteEligibility);

// =====
// 4. PRACTICE EXERCISES & BUG FIXES
// =====

console.log("\n--- Practice Questions ---");

// --- Question 3: Voting Eligibility with Input Validation ---
let userAge = 28;
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
if (userAge < 0) {
  console.log("Q3: Age must be a positive number.");
} else if (userAge >= 18) {
  console.log("Q3: Eligible to Vote");
} else {
  console.log("Q3: Not eligible to vote");
}

// --- Question 4: Largest of Two Numbers (Ternary) ---
let n1 = 23;
let n2 = 34;
let largerNum = (n1 > n2) ? n1 : n2;
console.log("Q4: Larger number is ->", largerNum);

// --- Question 5: Largest of Three Numbers using Math.max() ---
let num1 = 23;
let num2 = 34;
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
let num3 = 56;

let largestOfThree = Math.max(num1, num2, num3);
console.log("Q5: Largest of three numbers ->", largestOfThree);

// --- Question 6: Divisibility by both 3 and 5 ---
let testNumber1 = 15;
const divA1 = 3;
const divB1 = 5;

if (testNumber1 % divA1 === 0 && testNumber1 % divB1 === 0) {
  console.log("Q6: Number is divisible by BOTH 3 and 5.");
} else if (testNumber1 % divA1 === 0 || testNumber1 % divB1 === 0) {
  console.log("Q6: Number is divisible by ONLY ONE of the numbers.");
} else {
  console.log("Q6: Number is not divisible by any of the numbers.");
}
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
// --- Question 7: Divisibility by 5 and 10 ---
let testNumber2 = 15;
const fNum1 = 5;
const sNum1 = 10;

if (testNumber2 % fNum1 === 0 && testNumber2 % sNum1 === 0) {
  console.log("Q7: Number is divisible by BOTH 5 and 10.");
} else if (testNumber2 % fNum1 === 0 || testNumber2 % sNum1 === 0) {
  console.log("Q7: Number is divisible by ONLY ONE of the numbers (5 or 10).");
} else {
  console.log("Q7: Number is not divisible by any of the numbers.");
}

// --- Question 8: Student Grading System ---
let marks = 99;
let total = 100;
let studentGrade = "";

if (marks >= 0 && marks <= total) {
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
if (marks >= 90) {
  studentGrade = 'A+';
} else if (marks >= 70) {
  studentGrade = 'A';
} else if (marks >= 40) {
  studentGrade = 'B';
} else if (marks >= 30) {
  studentGrade = 'C';
} else {
  console.log("Q8: Student has Failed.");
}

if (studentGrade) {
  console.log("Q8: Student Grade is =>", studentGrade);
}
} else if (marks < 0) {
  console.log("Q8: Marks must be positive.");
} else {
  console.log("Q8: Marks cannot be greater than total marks!");
}
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
// --- Question 9: Categorize Person by Age (Child, Teenager, Adult) ---
let personAge = 0;

if (personAge <= 0) {
  console.log("Q9: Age must be a positive integer.");
} else if (personAge < 12) {
  console.log("Q9: Child");
} else if (personAge <= 18) {
  console.log("Q9: Teenager");
} else {
  console.log("Q9: Adult");
}

// --- Question 10: Check if Number is between 10 and 50 ---
let n3 = 34;

if (n3 >= 10 && n3 <= 50) {
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
        console.log("Q10:", n3, "is within the range of 10 to 50");
    } else {
        console.log("Q10:", n3, "is outside the range of 10 to 50");
    }

    // --- Question 11: Check Username and Password ---
    let inputUsername = "admin_user";
    let inputPassword = "SecurePassword123";

    const validUsername = "admin_user";
    const validPassword = "SecurePassword123";

    if (inputUsername === validUsername && inputPassword === validPassword) {
        console.log("Q11: Username and password are correct.");
    } else {
        console.log("Q11: Invalid username or password.");
    }
}
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
// --- Question 12: Restricted Area Access (Age 18+ and ID) ---
let visitorAge = 20;
let hasValidID = true;

if (visitorAge >= 18 && hasValidID) {
  console.log("Q12: Access granted to restricted area.");
} else {
  console.log("Q12: Access denied. Must be 18+ and possess an ID.");
}

// --- Question 13: Check if Number is Positive and Even ---
let checkNumPositiveEven = 14;

if (checkNumPositiveEven > 0 && checkNumPositiveEven % 2 === 0) {
  console.log("Q13:", checkNumPositiveEven, "is Positive and Even.");
} else {
  console.log("Q13:", checkNumPositiveEven, "is NOT both Positive and Even.");
}
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
// --- Question 14: Check if Number is Negative or Odd ---
let checkNumNegOdd = -7;

if (checkNumNegOdd < 0 || checkNumNegOdd % 2 !== 0) {
  console.log("Q14:", checkNumNegOdd, "is Negative or Odd.");
} else {
  console.log("Q14:", checkNumNegOdd, "is neither Negative nor Odd (It is Positive and Even).");
}

// --- Question 15: Temperature Classification (Hot, Normal, Cold) ---
let tempCelsius = 28;

if (tempCelsius > 35) {
  console.log("Q15: Temperature Classification -> Hot");
} else if (tempCelsius >= 15) {
  console.log("Q15: Temperature Classification -> Normal");
} else {
  console.log("Q15: Temperature Classification -> Cold");
}
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
}

// --- Question 16: Ticket Type via Ternary Operator ---
let ticketAge = 10;
let ticketType = (ticketAge < 12) ? "Child Ticket" : "Adult Ticket";
console.log("Q16: Ticket Assigned ->", ticketType);

// --- Question 17: Admin Panel Access Check ---
let isLoggedIn = true;
let isAdmin = true;

if (isLoggedIn && isAdmin) {
  console.log("Q17: Access granted to Admin Panel.");
} else {
  console.log("Q17: Access denied. User must be logged in AND have Admin status.");
}
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
// --- Question 18: Check if Passed All Subjects (Passing Marks = 35) ---
let mathMarks = 45;
let scienceMarks = 38;
let englishMarks = 50;
const passingMarks = 35;

if (mathMarks >= passingMarks && scienceMarks >= passingMarks && englishMarks >= passingMarks) {
  console.log("Q18: Student has passed all subjects.");
} else {
  console.log("Q18: Student has failed one or more subjects.");
}

// --- Question 19: Positive, Negative, or Zero Check ---
let valueToCheck = 0;

if (valueToCheck > 0) {
  console.log("Q19:", valueToCheck, "is Positive.");
} else if (valueToCheck < 0) {
  console.log("Q19:", valueToCheck, "is Negative.");
}
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
} else {
  console.log("Q19: Number is Zero.");
}

// --- Question 20: ATM Withdrawal Simulation ---
let correctPin = 4567;
let withdrawalAmount = 10000;
let accountBalance = 50000;

// Prompt execution conditional on environment availability
if (typeof prompt !== "undefined") {
  let enteredPin = Number(prompt("Enter PIN:"));

  if (enteredPin === correctPin) {
    console.log("Q20: PIN is correct.");

    if (withdrawalAmount > 0 && withdrawalAmount <= accountBalance) {
      accountBalance -= withdrawalAmount;
      console.log("Q20: Withdrawal successful! Withdrawn Amount:", withdrawalAmount);
    }
  }
}
```

# Clean Session Code

Source appendix - original Session 3 code formatted for reading.

---

```
        console.log("Q20: Remaining Balance:", accountBalance);
    } else if (withdrawalAmount > accountBalance) {
        console.log("Q20: Insufficient Balance!");
    } else {
        console.log("Q20: Invalid withdrawal amount!");
    }
} else {
    console.log("Q20: Pin is Incorrect. Cannot Withdraw.");
}
}
```